

Управление очередью методами обучения с подкреплением

Queue Control with Reinforcement Learning

Технический отчет по проекту

Публичная версия

Содержание

Введение	4
1. Теоретические основы обучения с подкреплением	6
1.1. Обучение с подкреплением	6
1.2. Политика, вознаграждение и функция ценности	6
1.3. Марковский процесс принятия решений	7
1.4. Уравнения Беллмана	7
1.5. Методы, основанные на модели, и безмодельные методы	8
2. Постановка задачи управления очередью	9
2.1. Описание системы	9
2.2. Множество состояний и действий	9
2.3. Случайный приход заявок	9
2.4. Случайное обслуживание	9
2.5. Динамика состояния	10
2.6. Функция вознаграждения	10
2.7. Целевой функционал	11
2.8. Пример таблицы переходов при малой емкости очереди	11
2.9. Используемые параметры модели	13
3. Решение задачи при известной модели	14
3.1. Переход к вычислительной модели	14
3.2. Метод итерации по ценности	14
3.3. Интерпретация функции ценности	15
3.4. Метод итерации по политикам	15
3.5. Сравнение двух методов	16
4. Методы Монте-Карло для безмодельного обучения	17
4.1. Переход к обучению по эпизодам	17
4.2. Оценивание методом Монте-Карло	17
4.3. Исследование и использование	18
4.4. Метод Монте-Карло с исследовательскими стартами	18
4.5. Метод Монте-Карло с ϵ -жадной политикой	18
4.6. Метод Монте-Карло с разделенными политиками	19
4.7. Метрики сравнения с эталонным решением	19
5. Вычислительные эксперименты и сравнение методов	21
5.1. Схема проведения экспериментов	21
5.2. Метод Монте-Карло с исследовательскими стартами	22
5.3. Метод Монте-Карло с ϵ -жадной политикой	25
5.4. Метод Монте-Карло с разделенными политиками	27

5.5. Итоговое сравнение методов	32
Заключение	36
Приложение А. Таблицы вычислительных экспериментов	38
А.1. Локальный перебор параметров метода Монте-Карло с исследовательскими стартами	38
А.2. Локальный перебор параметров метода Монте-Карло с ε -жадной политикой	38
А.3. Перебор параметров метода Монте-Карло с разделенными политиками .	39

Введение

Задачи управления очередями возникают в вычислительных системах, телекоммуникационных сетях, сервисных центрах и других системах массового обслуживания. В таких задачах требуется выбирать режим работы обслуживающего устройства: снижать затраты при небольшой очереди или ускорять обслуживание при накоплении заявок.

В работе рассматривается дискретная стохастическая модель управления очередью. Состоянием системы является длина очереди, а действием — выбор режима работы сервера: выключенный, обычный или ускоренный. Поступление заявок и результат обслуживания являются случайными, поэтому задача формулируется как марковский процесс принятия решений.

Для решения задачи используются методы обучения с подкреплением. Агентом является правило выбора режима работы сервера, а средой — система обслуживания с очередью. Цель агента состоит в выборе политики, минимизирующей ожидаемые суммарные затраты, связанные с длиной очереди и стоимостью обслуживания.

Актуальность работы связана с тем, что обучение с подкреплением позволяет решать задачи последовательного принятия решений в условиях случайности. В работе сравниваются два подхода: решение при известной модели переходов и обучение по траекториям без явного использования этой модели.

Цель работы — построить модель управления очередью и сравнить несколько методов обучения с подкреплением для нахождения политики управления сервером.

Для достижения цели решались следующие задачи:

1. сформулировать задачу как марковский процесс принятия решений;
2. задать состояния, действия, вероятности переходов и функцию вознаграждения;
3. реализовать методы динамического программирования для известной модели;
4. получить эталонную функцию ценности и эталонную политику методом итерации по ценности;
5. проверить согласованность решений, полученных методами итерации по ценности и итерации по политикам;
6. построить среду-генератор эпизодов, не раскрывающую алгоритмам модель переходов;
7. реализовать и сравнить методы Монте-Карло;
8. провести вычислительные эксперименты и сравнить найденные функции ценности и политики с эталонным решением.

При сравнении методов в качестве основной метрики используется ошибка приближения функции ценности относительно эталонного решения. Дополнительно рассматривается доля состояний, в которых найденная политика совпадает с политикой, полученной методом итерации по ценности.

Теоретическая часть основана на стандартных понятиях обучения с подкреплением: марковских процессах принятия решений, уравнениях Беллмана, методах динамического программирования и методах Монте-Карло. Экспериментальная часть основана на собственной программной реализации модели очереди и исследуемых алгоритмов.

Работа состоит из введения, пяти разделов, заключения и приложения.

1. Теоретические основы обучения с подкреплением

1.1. Обучение с подкреплением

Обучение с подкреплением применяется к задачам последовательного принятия решений. Агент взаимодействует со средой: наблюдает состояние, выбирает действие, получает вознаграждение и переходит в новое состояние. В отличие от обучения с учителем, правильные действия заранее не заданы.

Цель агента — максимизировать суммарное вознаграждение, поэтому учитываются не только немедленные результаты действий, но и их будущие последствия.

Взаимодействие происходит по шагам времени $t = 0, 1, 2, \dots$. На шаге t агент наблюдает состояние S_t , выбирает действие A_t , получает вознаграждение R_{t+1} и переходит в состояние S_{t+1} . Траектория имеет вид

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$$

В рассматриваемой задаче состояние — длина очереди, действие — режим работы сервера, а вознаграждение — отрицательная стоимость.

1.2. Политика, вознаграждение и функция ценности

Политика задаёт правило выбора действий. В детерминированном случае

$$a = \pi(s),$$

а в стохастическом случае

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

В данной работе вознаграждение является отрицательной величиной: оно учитывает штраф за длину очереди и стоимость режима обслуживания. Поэтому максимизация вознаграждения соответствует минимизации затрат.

Доход в бесконечной задаче с дисконтированием определяется формулой

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad (1)$$

где $\gamma \in (0, 1)$ — коэффициент дисконтирования.

Функция ценности состояния для политики π задаётся как

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s], \quad (2)$$

а функция ценности действий — как

$$q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]. \quad (3)$$

Функция ценности показывает, насколько выгодно находиться в состоянии s при дальнейшем следовании политике π . Функция ценности действий показывает ожидаемый доход после выбора действия a в состоянии s и дальнейшего следования той же политике.

1.3. Марковский процесс принятия решений

Марковский процесс принятия решений задаётся множеством состояний, множеством действий, вероятностями переходов и функцией вознаграждения. В работе используется обозначение

$$\mathcal{M} = (S, A, P, r, \gamma),$$

где S — множество состояний, A — множество действий, $P(s' \mid s, a)$ — вероятность перехода, $r(s, a)$ — ожидаемое немедленное вознаграждение, γ — коэффициент дисконтирования.

Марковское свойство означает, что распределение следующего состояния зависит только от текущего состояния и действия:

$$\mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a, S_{t-1}, A_{t-1}, \dots) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a).$$

Для очереди это означает, что следующая длина очереди определяется текущей длиной и выбранным режимом обслуживания.

1.4. Уравнения Беллмана

Для фиксированной политики π функция ценности удовлетворяет уравнению Беллмана:

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s) \left(r(s, a) + \gamma \sum_{s' \in S} P(s' \mid s, a) v_\pi(s') \right). \quad (4)$$

Оптимальная функция ценности

$$v_*(s) = \max_{\pi} v_\pi(s)$$

удовлетворяет уравнению оптимальности Беллмана:

$$v_*(s) = \max_{a \in A} \left(r(s, a) + \gamma \sum_{s' \in S} P(s' \mid s, a) v_*(s') \right). \quad (5)$$

После нахождения $v_*(s)$ оптимальная политика выбирается по правилу

$$\pi_*(s) \in \arg \max_{a \in A} \left(r(s, a) + \gamma \sum_{s' \in S} P(s' \mid s, a) v_*(s') \right). \quad (6)$$

В дальнейшем эта политика используется как эталонная.

1.5. Методы, основанные на модели, и безмодельные методы

Методы обучения с подкреплением делятся на методы, основанные на модели, и безмодельные методы. В первом случае известны $P(s' \mid s, a)$ и $r(s, a)$, поэтому можно применять динамическое программирование: итерацию по ценности и итерацию по политикам.

В безмодельных методах алгоритм не использует таблицу переходов, а обучается по траекториям

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

В работе сначала строится эталонное решение по известной модели. Затем та же система используется как среда-генератор эпизодов для методов Монте-Карло.

При сравнении безмодельных методов с эталонным решением важно учитывать не только совпадение найденной политики с оптимальной, но и точность приближения функции ценности. Поэтому в вычислительных экспериментах отдельно сравниваются оценки функции ценности и политики, найденные разными алгоритмами.

2. Постановка задачи управления очередью

2.1. Описание системы

Рассматривается дискретная система обслуживания с одной очередью и одним сервером. На каждом шаге в систему случайно поступают заявки, а сервер выбирает один из режимов работы. Управление состоит в том, чтобы выбирать режим обслуживания в зависимости от текущей длины очереди.

Основной компромисс задачи состоит в следующем: медленное обслуживание снижает затраты на сервер, но может приводить к росту очереди; ускоренный режим уменьшает очередь быстрее, но требует больших затрат. Поэтому требуется найти политику управления, учитывающую оба фактора.

2.2. Множество состояний и действий

Пусть s_t — длина очереди в начале шага t . Очередь имеет конечную вместимость N , поэтому

$$S = \{0, 1, \dots, N\}. \quad (7)$$

На каждом шаге выбирается один из трех режимов работы сервера:

$$A = \{\text{off}, \text{norm}, \text{fast}\}. \quad (8)$$

Действие `off` означает отсутствие обслуживания, `norm` — обычный режим, `fast` — ускоренный режим. Политика задает правило выбора режима:

$$a_t = \pi(s_t).$$

2.3. Случайный приход заявок

За один шаг в систему может прийти не более одной заявки. Обозначим число пришедших заявок через X_t :

$$X_t \in \{0, 1\}.$$

Вероятности прихода задаются параметром λ :

$$\mathbb{P}(X_t = 1) = \lambda, \quad \mathbb{P}(X_t = 0) = 1 - \lambda. \quad (9)$$

2.4. Случайное обслуживание

Число заявок, которое сервер пытается обработать на шаге t , обозначим через Y_t . Его распределение зависит от выбранного действия.

Для режима `off`:

$$Y_t = 0.$$

Для режима norm:

$$Y_t \in \{0, 1\}, \quad \mathbb{P}(Y_t = 1) = \mu, \quad \mathbb{P}(Y_t = 0) = 1 - \mu. \quad (10)$$

Для режима fast:

$$Y_t \in \{0, 1, 2\}, \quad (11)$$

$$\mathbb{P}(Y_t = 0) = p, \quad \mathbb{P}(Y_t = 1) = q, \quad \mathbb{P}(Y_t = 2) = 1 - p - q. \quad (12)$$

Фактически сервер не может обработать больше заявок, чем находится в очереди, поэтому число реально обслуженных заявок равно $\min(s_t, Y_t)$.

2.5. Динамика состояния

Внутри шага сначала происходит обслуживание, затем приход новой заявки:

обслуживание $\longrightarrow$ приход.

Следующее состояние задается формулой

$$s_{t+1} = \min\{N, s_t - \min(s_t, Y_t) + X_t\}. \quad (13)$$

Внутренний минимум учитывает, что нельзя обслужить больше заявок, чем есть в очереди, а внешний минимум отражает ограниченную емкость очереди.

Вероятность перехода имеет вид

$$P(s' \mid s, a) = \mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a). \quad (14)$$

В данной модели при достижении емкости очереди N новое состояние ограничивается сверху величиной N . Это означает, что при переполнении новая заявка фактически теряется, однако отдельный штраф за потерю заявки в текущей версии модели не вводится. Такой штраф можно было бы добавить в функцию вознаграждения, например в виде дополнительного отрицательного слагаемого при событии переполнения. В настоящей работе используется более простая постановка, где учитываются только длина очереди и стоимость режима обслуживания.

2.6. Функция вознаграждения

Вознаграждение за один шаг задается как отрицательная стоимость:

$$r(s, a) = -\alpha s - c(a), \quad (15)$$

где $\alpha > 0$ — вес штрафа за одну заявку в очереди, а $c(a)$ — стоимость выбранного режима обслуживания.

Стоимость режимов задается так:

$$c(\text{off}) = 0, \quad c(\text{norm}) = c_n, \quad c(\text{fast}) = c_f,$$

где

$$c_f > c_n > 0.$$

Таким образом, максимизация вознаграждения соответствует минимизации затрат, связанных с длиной очереди и стоимостью обслуживания.

2.7. Целевой функционал

Для политики π качество управления оценивается дисконтированной суммой вознаграждений:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad 0 < \gamma < 1. \quad (16)$$

Задача состоит в нахождении оптимальной политики

$$\pi^* \in \arg \max_{\pi} J(\pi). \quad (17)$$

Так как вознаграждение является отрицательной стоимостью, это эквивалентно минимизации ожидаемых суммарных дисконтированных затрат.

2.8. Пример таблицы переходов при малой емкости очереди

Для иллюстрации явно выпишем ненулевые вероятности переходов при $N = 4$. В этом случае

$$S = \{0, 1, 2, 3, 4\}.$$

Для краткости обозначим

$$r_{so} = r(s, \text{off}), \quad r_{sn} = r(s, \text{norm}), \quad r_{sf} = r(s, \text{fast}).$$

Таблицы ниже получаются из формулы (13) с учетом распределений X_t и Y_t .

Таблица 1: Переходы для режима off при $N = 4$.

s	s'	$P(s' \mid s, \text{off})$	$r(s, a)$
0	0	$1 - \lambda$	r_{0o}
0	1	λ	r_{0o}
1	1	$1 - \lambda$	r_{1o}
1	2	λ	r_{1o}
2	2	$1 - \lambda$	r_{2o}
2	3	λ	r_{2o}
3	3	$1 - \lambda$	r_{3o}
3	4	λ	r_{3o}
4	4	1	r_{4o}

Таблица 2: Переходы для режима norm при $N = 4$.

s	s'	$P(s' \mid s, \text{norm})$	$r(s, a)$
0	0	$1 - \lambda$	r_{0n}
0	1	λ	r_{0n}
1	0	$\mu(1 - \lambda)$	r_{1n}
1	1	$\mu\lambda + (1 - \mu)(1 - \lambda)$	r_{1n}
1	2	$(1 - \mu)\lambda$	r_{1n}
2	1	$\mu(1 - \lambda)$	r_{2n}
2	2	$\mu\lambda + (1 - \mu)(1 - \lambda)$	r_{2n}
2	3	$(1 - \mu)\lambda$	r_{2n}
3	2	$\mu(1 - \lambda)$	r_{3n}
3	3	$\mu\lambda + (1 - \mu)(1 - \lambda)$	r_{3n}
3	4	$(1 - \mu)\lambda$	r_{3n}
4	3	$\mu(1 - \lambda)$	r_{4n}
4	4	$1 - \mu(1 - \lambda)$	r_{4n}

Таблица 3: Переходы для режима fast при $N = 4$.

s	s'	$P(s' s, \text{fast})$	$r(s, a)$
0	0	$1 - \lambda$	r_{0f}
0	1	λ	r_{0f}
1	0	$(1 - p)(1 - \lambda)$	r_{1f}
1	1	$(1 - p)\lambda + p(1 - \lambda)$	r_{1f}
1	2	$p\lambda$	r_{1f}
2	0	$(1 - p - q)(1 - \lambda)$	r_{2f}
2	1	$(1 - p - q)\lambda + q(1 - \lambda)$	r_{2f}
2	2	$q\lambda + p(1 - \lambda)$	r_{2f}
2	3	$p\lambda$	r_{2f}
3	1	$(1 - p - q)(1 - \lambda)$	r_{3f}
3	2	$(1 - p - q)\lambda + q(1 - \lambda)$	r_{3f}
3	3	$q\lambda + p(1 - \lambda)$	r_{3f}
3	4	$p\lambda$	r_{3f}
4	2	$(1 - p - q)(1 - \lambda)$	r_{4f}
4	3	$(1 - p - q)\lambda + q(1 - \lambda)$	r_{4f}
4	4	$p + q\lambda$	r_{4f}

2.9. Используемые параметры модели

В вычислительных экспериментах используются фиксированные параметры модели: емкость очереди N , вероятность прихода λ , вероятность обслуживания в обычном режиме μ , вероятности p , q , $1 - p - q$ для ускоренного режима, а также стоимости $c(\text{off})$, $c(\text{norm})$, $c(\text{fast})$.

Стоимость режимов выбирается так, чтобы

$$c(\text{off}) < c(\text{norm}) < c(\text{fast}).$$

Это отражает предположение, что ускоренное обслуживание быстрее уменьшает очередь, но требует больших затрат.

Конкретные численные значения параметров задаются далее в разделе с вычислительными экспериментами.

3. Решение задачи при известной модели

3.1. Переход к вычислительной модели

После задания состояний, действий, вознаграждения и вероятностей переходов задача становится конечным марковским процессом принятия решений. В этом разделе предполагается, что модель известна полностью: для каждой пары (s, a) доступны ненулевые вероятности $P(s' | s, a)$ и значение $r(s, a)$.

В экспериментах использовались параметры

$$N = 50, \quad \gamma = 0.7, \quad \lambda = 0.8, \quad \mu = 0.7,$$

$$\mathbb{P}(Y_t = 0 | a_t = \text{fast}) = 0.1, \quad \mathbb{P}(Y_t = 1 | a_t = \text{fast}) = 0.3,$$

$$\mathbb{P}(Y_t = 2 | a_t = \text{fast}) = 0.6,$$

$$\alpha = 1.0, \quad c(\text{off}) = 0.0, \quad c(\text{norm}) = 0.5, \quad c(\text{fast}) = 1.5.$$

Критерий остановки итерационных алгоритмов:

$$\varepsilon_{\text{tol}} = 10^{-6}.$$

Таблица переходов строилась перебором всех возможных значений прихода x и обслуживания y . Для каждой пары (s, a) следующее состояние вычислялось по формуле

$$s' = \min\{N, s - \min(s, y) + x\}.$$

Если разные пары (x, y) приводили к одному состоянию s' , их вероятности суммировались.

3.2. Метод итерации по ценности

Метод итерации по ценности приближает оптимальную функцию ценности $v_*(s)$ с помощью оператора оптимальности Беллмана. Начальное приближение задавалось как

$$V_0(s) = 0, \quad s \in S.$$

На каждой итерации выполнялось обновление

$$V_{k+1}(s) = \max_{a \in A} \left(r(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) V_k(s') \right). \quad (18)$$

Итерации продолжались до выполнения условия

$$\max_{s \in S} |V_{k+1}(s) - V_k(s)| < \varepsilon_{\text{tol}}. \quad (19)$$

После сходимости политика восстанавливалась жадным выбором действия:

$$\pi_{VI}(s) \in \arg \max_{a \in A} \left(r(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) V(s') \right). \quad (20)$$

В дальнейшем функция ценности V_{VI} и политика π_{VI} , полученные методом итерации по ценности, используются как эталонные.

3.3. Интерпретация функции ценности

Так как вознаграждение является отрицательной стоимостью, функция $V(s)$ имеет смысл ожидаемой суммарной отрицательной стоимости при оптимальном управлении. При росте длины очереди состояние становится хуже, поэтому значение функции ценности убывает.

Оптимальная политика имеет пороговый характер: при малой очереди достаточно режима off или norm, а при большой очереди выгоднее использовать fast, несмотря на его большую стоимость.

3.4. Метод итерации по политикам

Для проверки результата был реализован метод итерации по политикам. Он чередует два шага: оценивание текущей политики и ее улучшение.

Пусть задана детерминированная политика π_k . На этапе оценивания вычисляется функция ценности этой политики:

$$v_{\pi_k}(s) = r(s, \pi_k(s)) + \gamma \sum_{s' \in S} P(s' | s, \pi_k(s)) v_{\pi_k}(s'). \quad (21)$$

В реализации она находилась итерационно:

$$V_{m+1}(s) = r(s, \pi_k(s)) + \gamma \sum_{s' \in S} P(s' | s, \pi_k(s)) V_m(s'). \quad (22)$$

Итерации продолжались до условия

$$\max_{s \in S} |V_{m+1}(s) - V_m(s)| < \varepsilon_{tol}.$$

После этого политика улучшалась:

$$\pi_{k+1}(s) \in \arg \max_{a \in A} \left(r(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) v_{\pi_k}(s') \right). \quad (23)$$

Если политика не менялась, алгоритм останавливался.

3.5. Сравнение двух методов

Метод итерации по ценности напрямую приближает оптимальную функцию ценности, а метод итерации по политикам чередует оценивание и улучшение политики. Оба метода используют одну и ту же полную модель переходов.

В рассматриваемой задаче методы дали одинаковую оптимальную политику. Это служит проверкой корректности построенной таблицы переходов, функции вознаграждения и программной реализации.

Далее в качестве эталонного решения используется результат метода итерации по ценности: функция ценности V_{VI} и политика π_{VI} . Именно с ними сравниваются безмодельные методы Монте-Карло: по ошибке приближения функции ценности и по совпадению найденной политики с эталонной.

4. Методы Монте-Карло для безмодельного обучения

4.1. Переход к обучению по эпизодам

В этом разделе задача рассматривается в безмодельной постановке. Алгоритм не использует таблицу вероятностей переходов $P(s' | s, a)$, а обучается по траекториям взаимодействия со средой.

Для этого была реализована среда `QueueControlEnv`. Она моделирует динамику очереди и для алгоритмов выступает как «черный ящик»: на выбранное действие среда возвращает следующее состояние и вознаграждение.

Один шаг взаимодействия имеет вид

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

Эпизод задается конечной последовательностью таких переходов:

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T, S_T.$$

Так как исходная задача бесконечная, в вычислительных экспериментах эпизоды ограничиваются максимальной длиной.

4.2. Оценивание методом Монте-Карло

Методы Монте-Карло оценивают ожидаемые доходы по выборочным траекториям. Для момента времени t доход в конечном эпизоде определяется как

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T. \quad (24)$$

Он вычисляется обратным проходом по эпизоду:

$$G \leftarrow \gamma G + R_{t+1}. \quad (25)$$

В задачах управления оценивается функция ценности действий $q(s, a)$. Ее табличная оценка обозначается через $Q(s, a)$. По этим оценкам строится жадная политика:

$$\pi(s) \in \arg \max_{a \in A} Q(s, a).$$

В работе используются инкрементные обновления:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (G - Q(s, a)), \quad (26)$$

где $\alpha > 0$ — параметр размера шага.

4.3. Исследование и использование

При обучении необходимо совмещать использование уже найденных хороших действий и исследование альтернативных действий. Если всегда выбирать только текущее лучшее действие, алгоритм может закрепиться на неоптимальной политике, особенно в начале обучения.

В работе сравниваются три метода Монте-Карло:

1. метод с исследовательскими стартами;
2. метод с единой ε -жадной политикой;
3. метод с разделенными политиками и взвешенной выборкой по значимости.

Все методы обучаются на эпизодах, сгенерированных средой `QueueControlEnv`.

4.4. Метод Монте-Карло с исследовательскими стартами

В методе с исследовательскими стартами начальная пара (S_0, A_0) выбирается случайно. Это позволяет получать информацию о разных действиях даже в тех состояниях, куда текущая политика приводит редко.

После начального действия дальнейшее поведение определяется текущей жадной политикой:

$$\pi(s) \in \arg \max_{a \in A} Q(s, a).$$

После завершения эпизода выполняется обратный проход, оценки $Q(s, a)$ обновляются по формуле (26), а политика улучшается:

$$\pi(s) \leftarrow \arg \max_{a \in A} Q(s, a). \quad (27)$$

В реализации использовалось обновление по всем посещениям пар «состояние–действие»: если пара (s, a) встречалась в эпизоде несколько раз, то обновление выполнялось для каждого посещения.

4.5. Метод Монте-Карло с ε -жадной политикой

Во втором методе действия выбираются по ε -жадному правилу:

$$A_t = \begin{cases} \arg \max_{a \in A} Q(S_t, a), & \text{с вероятностью } 1 - \varepsilon, \\ \text{случайное действие из } A, & \text{с вероятностью } \varepsilon. \end{cases} \quad (28)$$

Такой выбор обеспечивает постоянное исследование действий. В реализации также использовался случайный выбор начального состояния, чтобы разные части пространства состояний посещались чаще. После генерации эпизода доходы вычисляются обратным проходом, а оценки $Q(s, a)$ обновляются по формуле (26). Итоговая политика затем определяется жадно по найденным оценкам.

4.6. Метод Монте-Карло с разделенными политиками

Третий метод относится к off-policy методам Монте-Карло. В нем различаются две политики:

- поведенческая политика b , по которой генерируются эпизоды;
- целевая политика π , которую нужно оценить и улучшить.

В работе поведенческая политика b выбирается ε -жадной, чтобы сохранялось исследование действий. Целевая политика π является жадной по текущим оценкам функции качества:

$$\pi(s) \in \arg \max_{a \in A} Q(s, a).$$

Так как эпизоды генерируются по политике b , а оценивается целевая политика π , необходимо учитывать различие между ними. Для этого используется отношение вероятностей траектории:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}. \quad (29)$$

Обычная выборка по значимости может иметь большую дисперсию, поэтому в работе использовалась взвешенная выборка по значимости. Для каждой пары (s, a) накапливался суммарный вес $C(s, a)$:

$$C(s, a) \leftarrow C(s, a) + W, \quad (30)$$

а оценка обновлялась по правилу

$$Q(s, a) \leftarrow Q(s, a) + \frac{W}{C(s, a)} (G - Q(s, a)), \quad (31)$$

где W — текущий вес выборки по значимости.

Так как целевая политика детерминированная, при несовпадении действия с целевой политикой обратный проход прерывается:

$$A_t \neq \pi(S_t).$$

Эта особенность важна для дальнейшего анализа результатов: если траектория часто отклоняется от целевой политики, значительная часть эпизода не используется при обновлении оценок $Q(s, a)$.

4.7. Метрики сравнения с эталонным решением

После обучения каждого метода по таблице оценок $Q(s, a)$ строилась приближенная функция ценности

$$V_{MC}(s) = \max_{a \in A} Q(s, a). \quad (32)$$

В качестве эталона использовалась оптимальная функция ценности $V_{VI}(s)$, найденная методом итерации по ценности. Основной метрикой качества была выбрана сумма квадратов отклонений:

$$SSE_V = \sum_{s \in S} (V_{MC}(s) - V_{VI}(s))^2. \quad (33)$$

Чем меньше значение SSE_V , тем точнее метод Монте-Карло приближает эталонную функцию ценности. Эта метрика является основной, так как она учитывает численные значения функции ценности во всех состояниях.

Дополнительно по таблице $Q(s, a)$ строилась жадная политика

$$\pi_{MC}(s) \in \arg \max_{a \in A} Q(s, a).$$

Для нее вычислялась доля состояний, в которых она совпадает с эталонной политикой π_{VI} :

$$\text{policy_match} = \frac{1}{|S|} \sum_{s \in S} \mathbf{1}\{\pi_{MC}(s) = \pi_{VI}(s)\}. \quad (34)$$

Метрика `policy_match` показывает совпадение выбранных действий, но не учитывает точность самих численных оценок функции ценности. Поэтому в итоговом сравнении основной считается метрика SSE_V , а `policy_match` используется как дополнительный показатель.

5. Вычислительные эксперименты и сравнение методов

5.1. Схема проведения экспериментов

Цель вычислительных экспериментов состояла в том, чтобы исследовать работу безмодельных методов Монте-Карло в задаче управления очередью и сравнить полученные результаты с эталонным решением, найденным при известной модели.

В качестве эталона использовались оптимальная функция ценности $V_{VI}(s)$ и политика π_{VI} , полученные методом итерации по ценности. В разделе 3 было показано, что метод итерации по политикам дал ту же оптимальную политику, поэтому найденное решение используется как эталонное.

Для каждого метода Монте-Карло проводилась серия запусков с различными значениями параметров и разными случайными зернами. После обучения по таблице оценок $Q(s, a)$ строилась приближенная функция ценности

$$V_{MC}(s) = \max_{a \in A} Q(s, a).$$

Затем вычислялась ошибка относительно эталонной функции ценности:

$$SSE_V = \sum_{s \in S} (V_{MC}(s) - V_{VI}(s))^2.$$

Эта метрика была выбрана основной. Чем меньше значение SSE_V , тем точнее метод Монте-Карло приближает функцию ценности, найденную методом итерации по ценности.

Дополнительно по оценкам $Q(s, a)$ строилась жадная политика

$$\pi_{MC}(s) \in \arg \max_{a \in A} Q(s, a).$$

Для нее вычислялась доля состояний, в которых она совпадает с эталонной политикой π_{VI} :

$$\text{policy_match} = \frac{1}{|S|} \sum_{s \in S} \mathbf{1}\{\pi_{MC}(s) = \pi_{VI}(s)\}.$$

Эта метрика показывает совпадение выбранных действий, но не учитывает точность самих численных значений функции ценности. Поэтому далее `policy_match` используется как дополнительная метрика.

Таким образом, в экспериментах анализировались обе характеристики: численная близость функции ценности к эталонной и совпадение найденной политики с политикой Value Iteration. Итоговая конфигурация каждого метода выбиралась в первую очередь по минимальному значению SSE_V . При близких значениях основной метрики дополнительно учитывались `policy_match`, устойчивость результата и время обучения.

Также для каждого запуска измерялось время обучения. Для сравнения конфигураций использовались средние значения метрик по нескольким случайным запускам, а также стандартные отклонения, характеризующие устойчивость метода.

5.2. Метод Монте-Карло с исследовательскими стартами

Первым был исследован метод Монте-Карло с исследовательскими стартами. В этом методе начальная пара (S_0, A_0) выбирается случайно, что позволяет получать информацию о разных действиях в разных состояниях. После первого шага дальнейшее поведение задается текущей жадной политикой, построенной по оценкам $Q(s, a)$.

Сначала исследовалось влияние числа эпизодов при фиксированных параметрах α и $\max_steps$. Увеличение числа эпизодов в среднем улучшает качество, так как алгоритм получает больше траекторий для обновления оценок. Однако после некоторого момента выигрыш становится меньше, а время обучения продолжает расти.

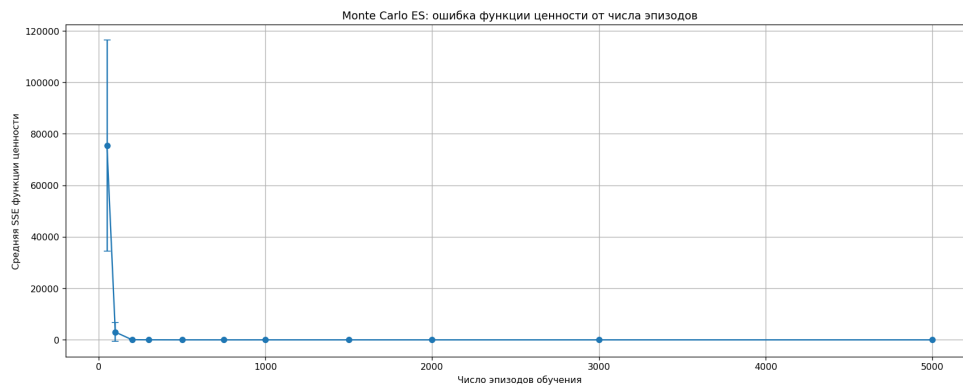


Рис. 1: Метод Монте-Карло с исследовательскими стартами: зависимость ошибки функции ценности от числа эпизодов.

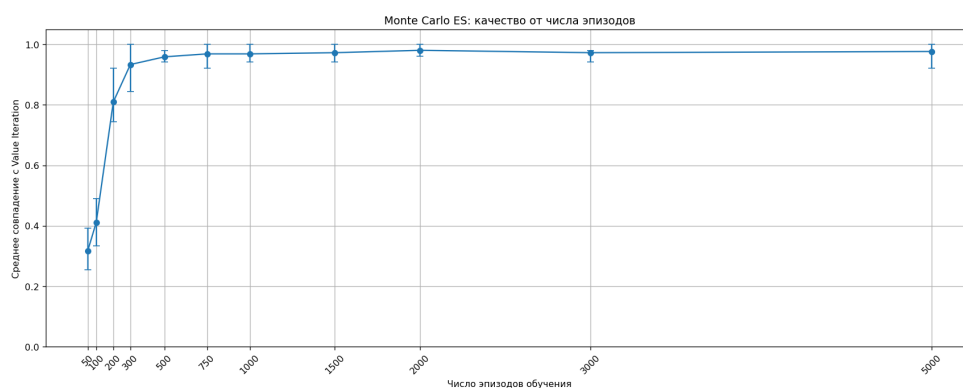


Рис. 2: Метод Монте-Карло с исследовательскими стартами: зависимость совпадения политик от числа эпизодов.

Графики показывают, что увеличение числа эпизодов влияет не только на ошибку функции ценности, но и на совпадение итоговой политики с эталонной. При этом значение `policy_match` быстрее выходит на высокий уровень, чем стабилизируются сами численные оценки функции ценности. Это естественно: для совпадения политик

достаточно правильно определить лучшее действие, тогда как метрика SSE_V требует точного приближения значений $V_{VI}(s)$.

Далее исследовалось влияние параметра размера шага α . Слишком малые значения α приводят к медленному изменению оценок, а слишком большие могут усиливать разброс результатов. Поэтому параметр α подбирался экспериментально.

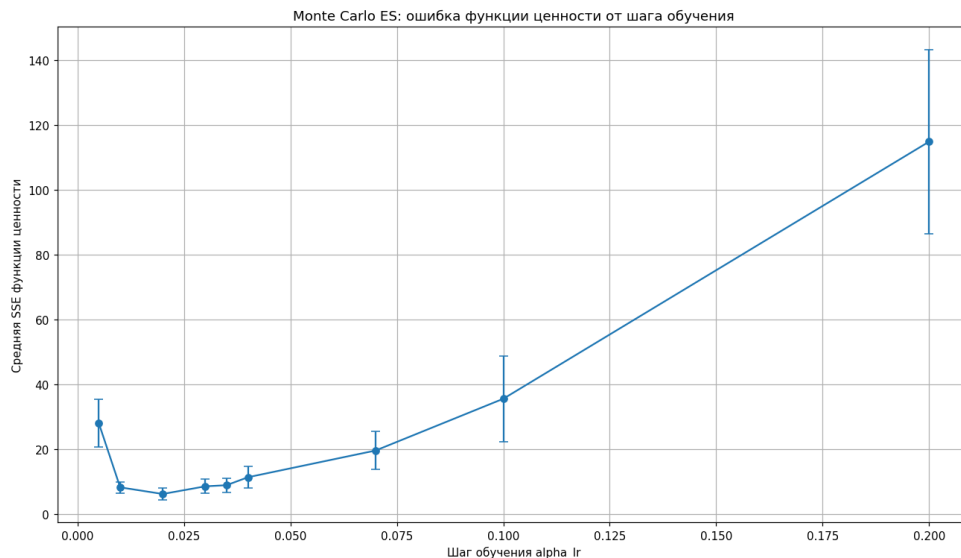


Рис. 3: Метод Монте-Карло с исследовательскими стартами: зависимость ошибки функции ценности от параметра α .

Также исследовалось влияние максимальной длины эпизода. Увеличение длины эпизода позволяет учитывать более длинные траектории, но одновременно увеличивает время обучения. Кроме того, слишком длинные эпизоды не всегда улучшают качество оценок.

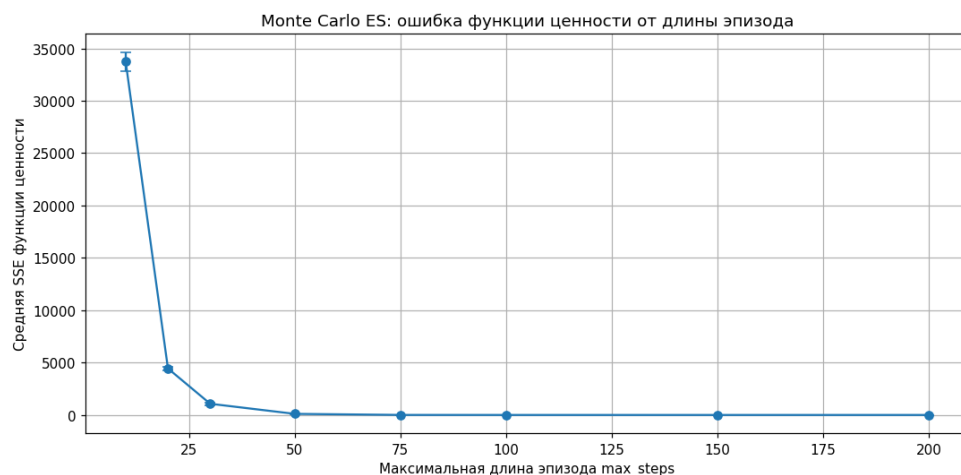


Рис. 4: Метод Монте-Карло с исследовательскими стартами: зависимость ошибки функции ценности от длины эпизода.

После отдельных экспериментов был выполнен локальный перебор параметров. Конфигурации сравнивались по основной метрике SSE_V , а `policy_match` использовалась для дополнительной проверки качества найденной политики.

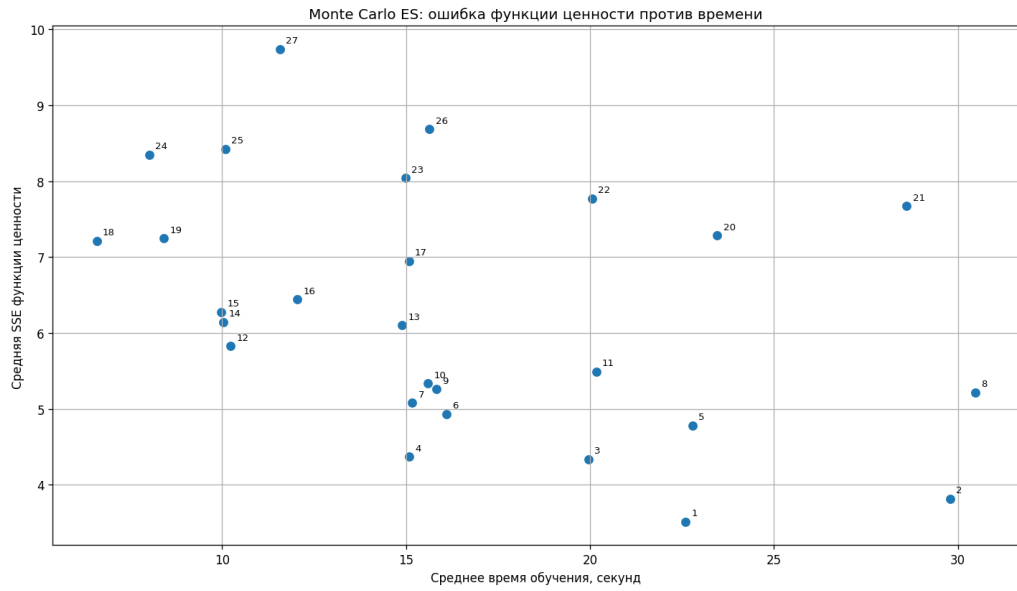


Рис. 5: Метод Монте-Карло с исследовательскими стартами: локальный перебор параметров по ошибке функции ценности и времени обучения.

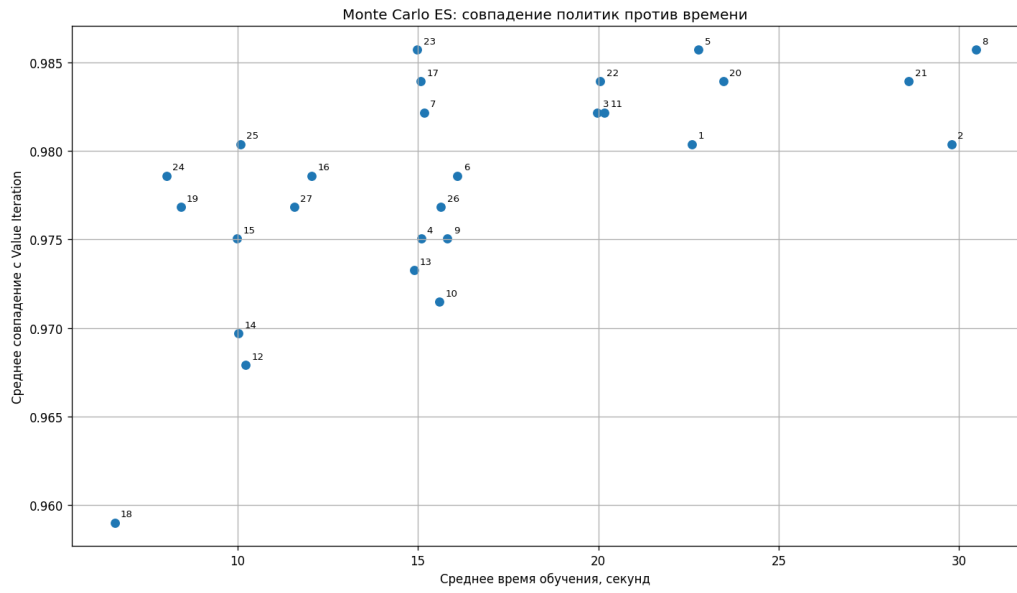


Рис. 6: Метод Монте-Карло с исследовательскими стартами: локальный перебор параметров по совпадению политик и времени обучения.

По результатам локального перебора лучшая конфигурация по основной метрике SSE_V была получена при

$$\text{num_episodes} = 3000, \quad \alpha = 0.015, \quad \text{max_steps} = 150.$$

Для этой конфигурации

$$\text{mean_value_sse} \approx 3.51,$$

а среднее совпадение с эталонной политикой составило

$$\text{mean_policy_match} \approx 0.980.$$

Среднее время обучения было примерно

22.60 секунды.

Значение `policy_match` показывает, что найденная политика близка к эталонной: она совпадает с политикой Value Iteration примерно в 98% состояний. Однако для итогового выбора используется именно значение SSE_V , так как оно отражает не только выбор действий, но и точность приближения функции ценности.

5.3. Метод Монте-Карло с ε -жадной политикой

Вторым был исследован метод Монте-Карло с единой ε -жадной политикой. В этом методе одна и та же политика используется для генерации эпизодов, а итоговая политика затем строится жадно по полученным оценкам. В каждом состоянии с вероятностью $1 - \varepsilon$ выбирается действие с наибольшей текущей оценкой, а с вероятностью ε выбирается случайное действие.

Сначала исследовалось влияние параметра ε . Этот параметр отвечает за баланс между использованием текущих оценок и исследованием альтернативных действий. При больших значениях ε алгоритм чаще выбирает случайные действия, что усиливает исследование, но может ухудшать точность итоговых оценок. При малых значениях ε алгоритм чаще следует текущей жадной политике.

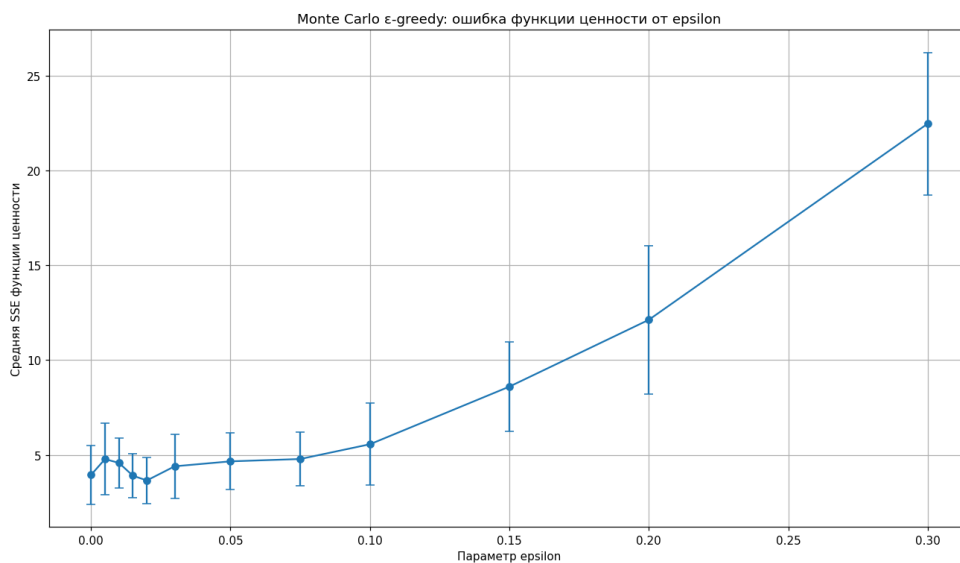


Рис. 7: Метод Монте-Карло с ε -жадной политикой: зависимость ошибки функции ценности от параметра ε .

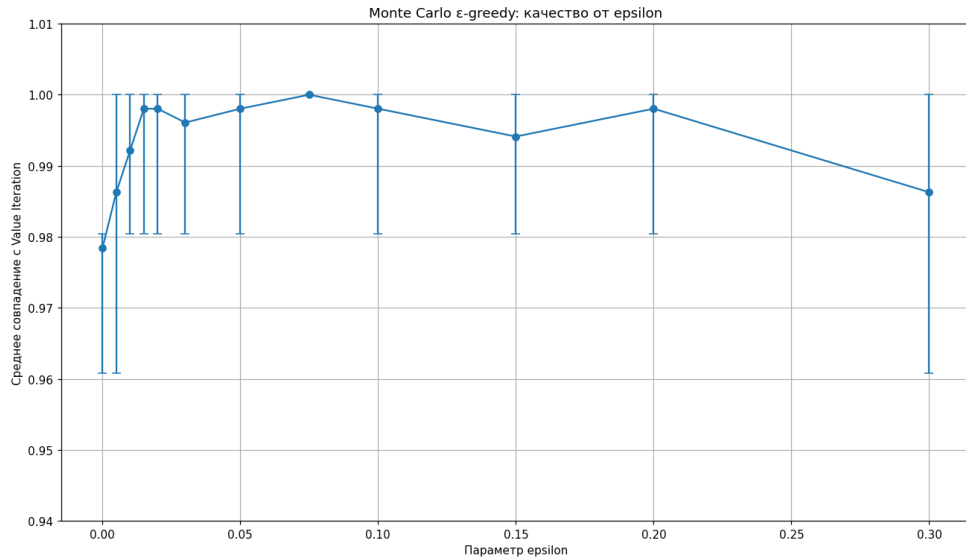


Рис. 8: Метод Монте-Карло с ε -жадной политикой: зависимость совпадения политик от параметра ε .

В проведенных экспериментах малая, но ненулевая вероятность случайного действия оказалась полезной. Значение $\varepsilon = 0.01$ позволило сохранить исследование действий и при этом не вносить слишком большой шум в оценки $Q(s, a)$. При этом метрика `policy_match` также оставалась высокой, то есть улучшение по ошибке функции ценности не происходило за счет ухудшения итоговой политики.

После этого был выполнен локальный перебор параметров. По основной метрике SSE_V лучшая конфигурация была получена при

$$\text{num_episodes} = 3000, \quad \alpha = 0.01, \quad \varepsilon = 0.01, \quad \text{max_steps} = 150.$$

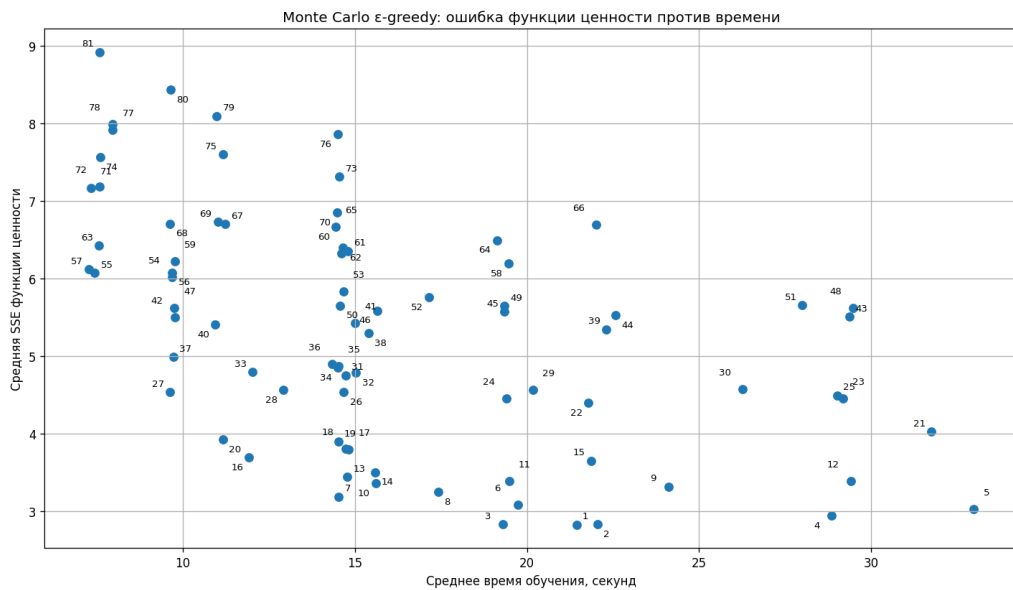


Рис. 9: Метод Монте-Карло с ε -жадной политикой: локальный перебор параметров по ошибке функции ценности и времени обучения.

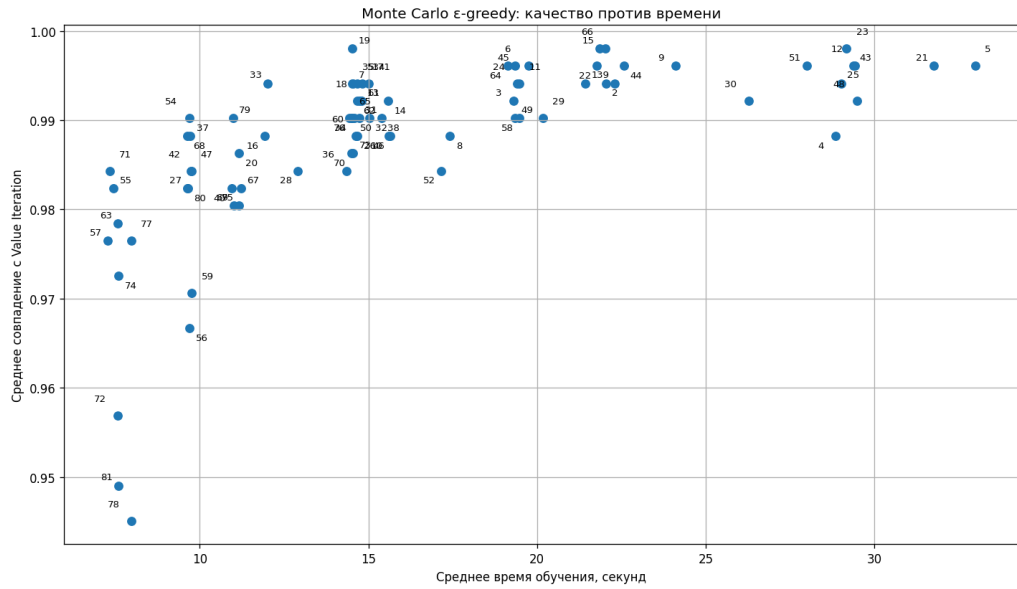


Рис. 10: Метод Монте-Карло с ϵ -жадной политикой: локальный перебор параметров по совпадению политик и времени обучения.

Для выбранной конфигурации

$$\text{mean_value_sse} \approx 2.83.$$

Среднее совпадение с эталонной политикой составило

$$\text{mean_policy_match} \approx 0.994.$$

Среднее время обучения было примерно

$$21.44 \text{ секунды.}$$

Для данного метода лучшие конфигурации по ошибке функции ценности и по совпадению политик оказались близкими. Это говорит о том, что найденные оценки $Q(s, a)$ не только дают правильный порядок действий в большинстве состояний, но и достаточно хорошо приближают численные значения функции ценности. Поэтому для итогового сравнения метода Монте-Карло с ϵ -жадной политикой была выбрана конфигурация

$$\text{num_episodes} = 3000, \quad \alpha = 0.01, \quad \epsilon = 0.01, \quad \text{max_steps} = 150.$$

5.4. Метод Монте-Карло с разделенными политиками

Третьим был исследован метод Монте-Карло с разделенными политиками и взвешенной выборкой по значимости. В этом методе различаются две политики: поведенческая политика b , по которой генерируются эпизоды, и целевая политика π , которую требуется оценить и улучшить.

Так как целевая политика является жадной и детерминированной, при несовпадении действия поведенческой политики с действием целевой политики обратный проход по эпизоду прерывается. Поэтому метод может использовать только часть сгенерированной траектории. Эта особенность оказалась важной для рассматриваемой задачи.

Сначала был выполнен пробный запуск метода. Несмотря на большое число эпизодов, качество оказалось заметно хуже, чем у двух предыдущих методов. Чтобы понять причину, была проведена диагностика работы алгоритма. При длине эпизода 100 среднее число реально использованных обновлений было только около 5, а условие прерывания обратного прохода срабатывало во всех эпизодах. Это означает, что большая часть собранных траекторий не использовалась для обновления оценок $Q(s, a)$.

Затем исследовалось влияние параметра ε поведенческой политики. При этом фиксировались

$$\text{num_episodes} = 20000, \quad \text{max_steps} = 100.$$

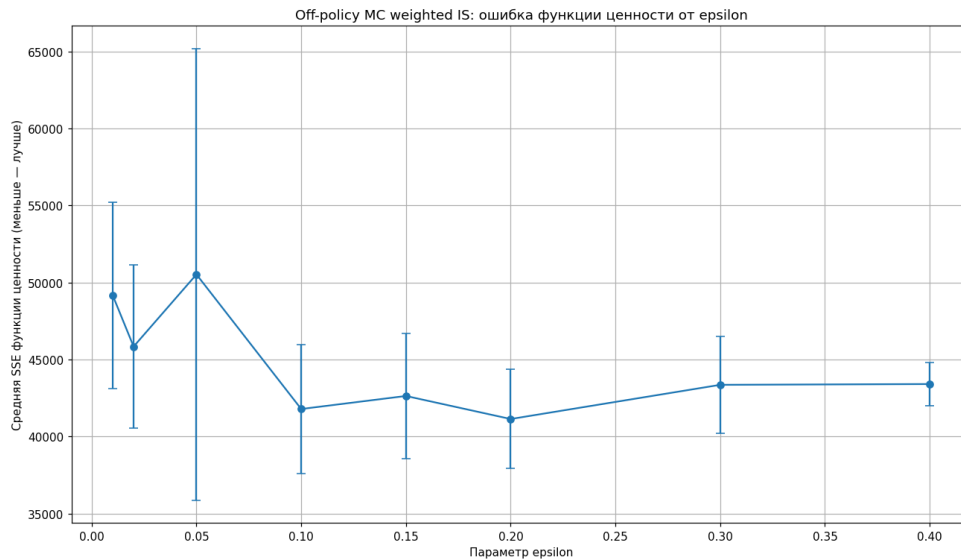


Рис. 11: Метод Монте-Карло с разделенными политиками: зависимость ошибки функции ценности от параметра ε .

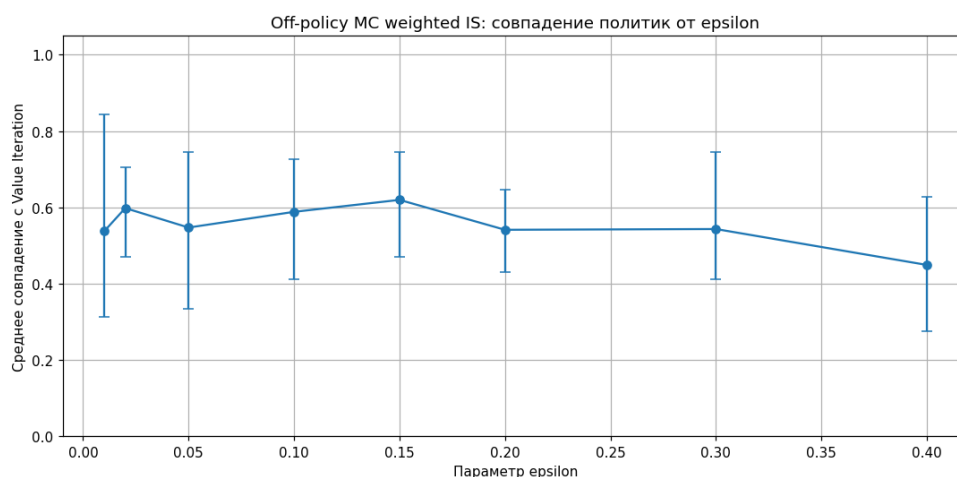


Рис. 12: Метод Монте-Карло с разделенными политиками: зависимость совпадения политик от параметра ε .

Изменение параметра ε не позволило получить качество, сравнимое с двумя предыдущими методами. Значения ошибки функции ценности оставались большими, а совпадение с эталонной политикой — низким. Это означает, что проблема метода проявляется не только в неточности численных оценок $V_{MC}(s)$, но и в самой найденной политике.

После этого проверялось влияние максимальной длины эпизода. Уменьшение длины эпизода позволяет сократить время обучения, но не устраняет главную проблему метода: обратный проход все равно часто обрывается из-за несовпадения действий поведенческой и целевой политик.

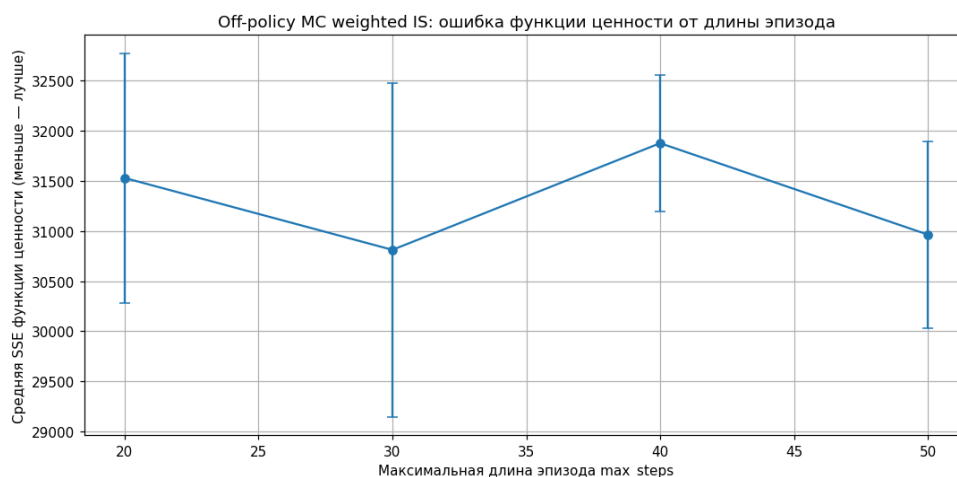


Рис. 13: Метод Монте-Карло с разделенными политиками: зависимость ошибки функции ценности от длины эпизода.

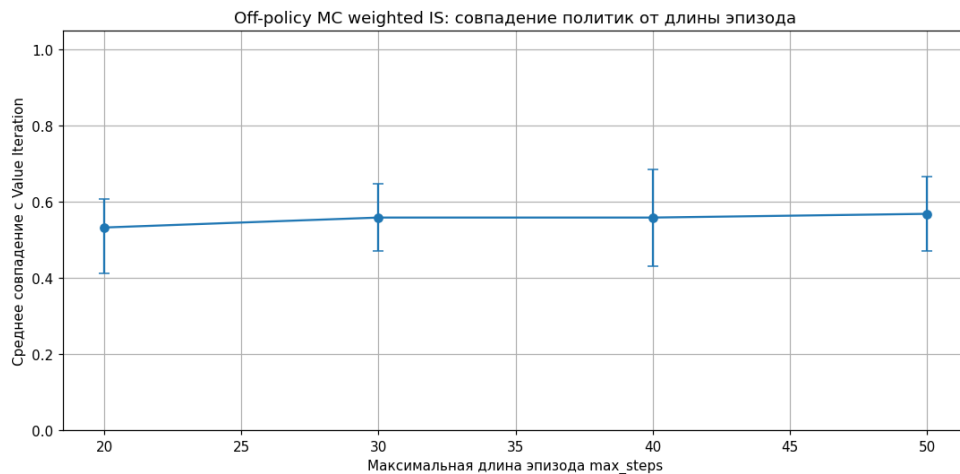


Рис. 14: Метод Монте-Карло с разделенными политиками: зависимость совпадения политик от длины эпизода.

Затем был выполнен грубый перебор параметров. В него были включены значения числа эпизодов, параметра ϵ и максимальной длины эпизода, оказавшиеся наиболее разумными по результатам предварительных экспериментов.

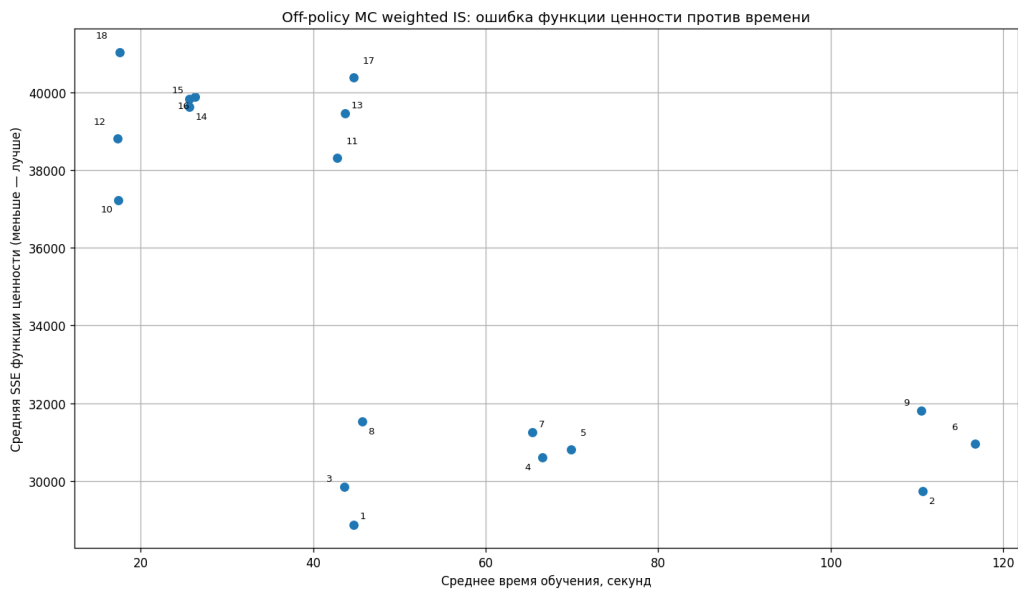


Рис. 15: Метод Монте-Карло с разделенными политиками: грубый перебор параметров по ошибке функции ценности и времени обучения.

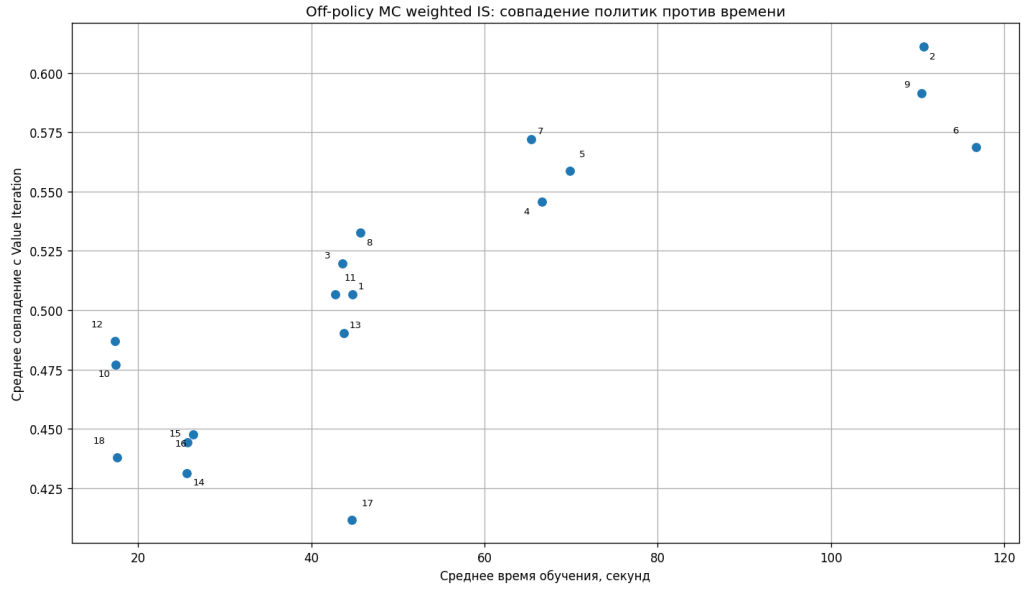


Рис. 16: Метод Монте-Карло с разделенными политиками: грубый перебор параметров по совпадению политик и времени обучения.

По результатам перебора лучшая конфигурация по основной метрике SSE_V была получена при

$$\text{num_episodes} = 50000, \quad \varepsilon = 0.15, \quad \text{max_steps} = 20.$$

Для нее

$$\text{mean_value_sse} \approx 28874.12,$$

а среднее совпадение с эталонной политикой составило

$$\text{mean_policy_match} \approx 0.507.$$

Среднее время обучения было примерно

$$44.73 \text{ секунды.}$$

Если ориентироваться только на совпадение политик, можно получить другую конфигурацию с более высоким значением policy_match , однако ошибка функции ценности при этом остается очень большой. Поэтому по основной метрике метод с разделенными политиками существенно уступает двум другим методам Монте-Карло. Низкое значение policy_match дополнительно подтверждает, что метод плохо восстанавливает и саму управляющую политику.

Полученный результат согласуется с диагностикой алгоритма. Из-за различия поведенческой и целевой политик обратный проход по эпизоду часто обрывается, поэтому значительная часть траектории не используется для обновления оценок. Для рассматриваемой задачи управления очередью этот метод оказался менее подходящим.

5.5. Итоговое сравнение методов

После подбора параметров для каждого метода были выбраны итоговые конфигурации. Их параметры приведены в таблице 4, а результаты сравнения — в таблице 5.

Таблица 4: Выбранные параметры методов Монте-Карло

Метод	Число эпизодов	α	ε	Длина эпизода
Метод Монте-Карло с исследовательскими стартами	3000	0.015	—	150
Метод Монте-Карло с ε -жадной политикой	3000	0.01	0.01	150
Метод Монте-Карло с разделенными политиками	50000	—	0.15	20

Таблица 5: Итоговое сравнение методов Монте-Карло

Метод	mean_value_sse	mean_policy_match	Время, сек.
Метод Монте-Карло с ε -жадной политикой	2.829	0.994	21.44
Метод Монте-Карло с исследовательскими стартами	3.514	0.980	22.60
Метод Монте-Карло с разделенными политиками	28874.12	0.507	44.73

Из таблицы видно, что метод Монте-Карло с разделенными политиками и weighted importance sampling оказался существенно хуже двух остальных методов. Его ошибка функции ценности на несколько порядков больше, а совпадение найденной политики с политикой Value Iteration составляет только около 0.507. Это означает, что метод восстанавливает эталонную политику примерно только в половине состояний. Поэтому далее отдельно сравниваются два метода, показавшие хорошее качество: метод Монте-Карло с исследовательскими стартами и метод Монте-Карло с единой ε -жадной политикой.

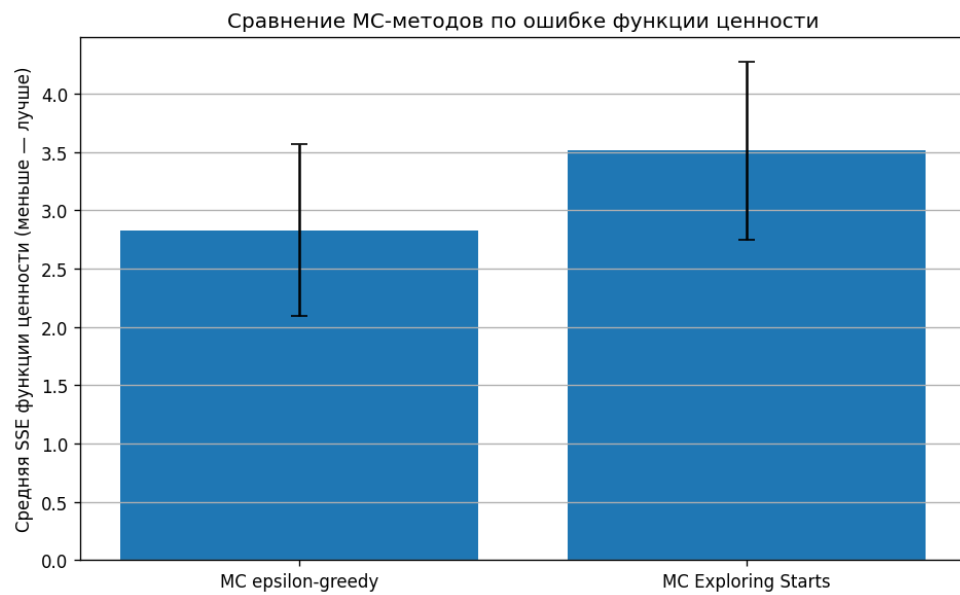


Рис. 17: Итоговое сравнение двух лучших методов Монте-Карло по ошибке функции ценности.

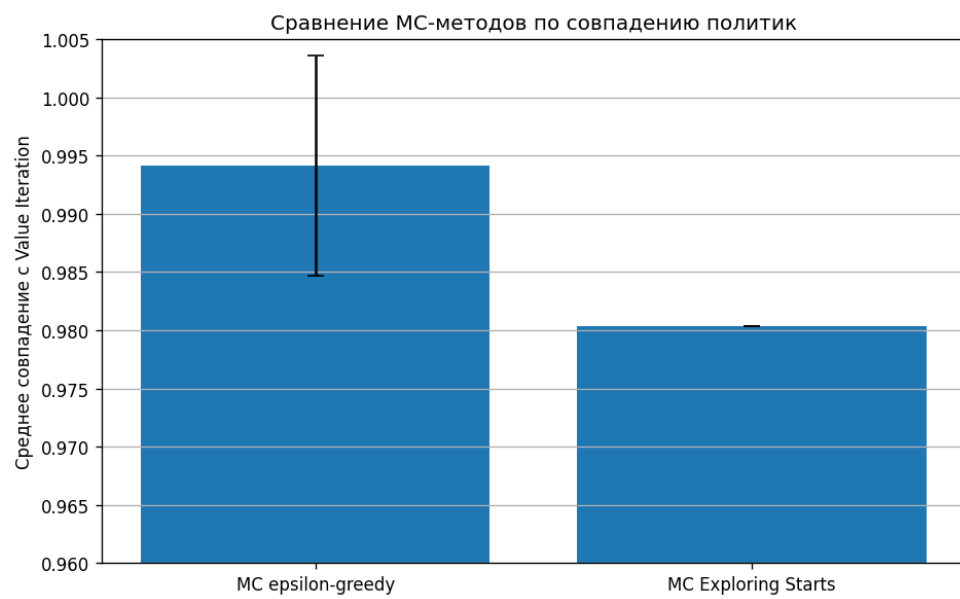


Рис. 18: Итоговое сравнение двух лучших методов Монте-Карло по совпадению политик.

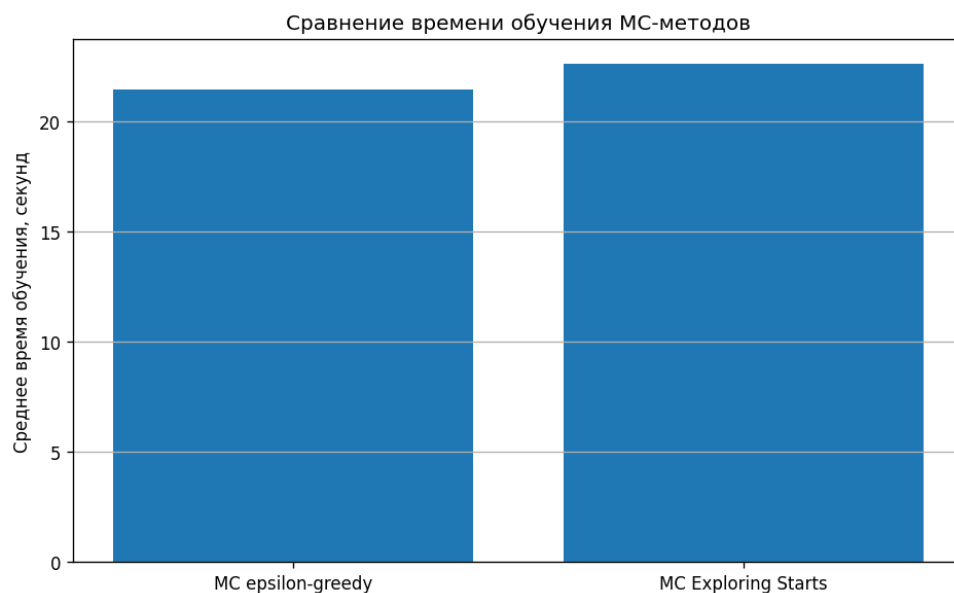


Рис. 19: Сравнение времени обучения двух лучших методов Монте-Карло.

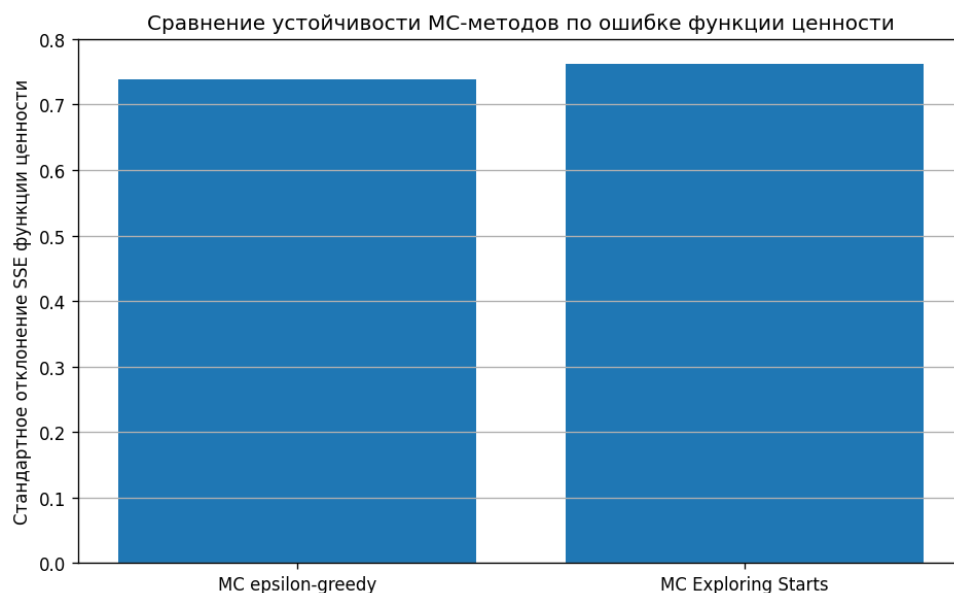


Рис. 20: Сравнение устойчивости двух лучших методов Монте-Карло по ошибке функции ценности.

Итоговое сравнение показывает, что два метода — Монте-Карло с исследовательскими стартами и Монте-Карло с ϵ -жадной политикой — дают близкие, но не одинаковые результаты. Лучший результат по основной метрике SSE_V показал метод с ϵ -жадной политикой:

$$\text{mean_value_sse} \approx 2.83.$$

Для метода с исследовательскими стартами средняя ошибка функции ценности составила

$$\text{mean_value_sse} \approx 3.51.$$

Таким образом, метод с ϵ -жадной политикой уменьшил среднюю ошибку функции цен-

ности на

$$3.51 - 2.83 \approx 0.68,$$

что соответствует относительному уменьшению ошибки примерно на

$$19.48\%.$$

По дополнительной метрике совпадения политик метод с ε -жадной политикой также оказался лучше:

$$0.994 > 0.980.$$

Это означает, что он чаще выбирает те же действия, что и эталонная политика, полученная методом итерации по ценности.

Время обучения двух лучших методов также близко. Метод с ε -жадной политикой оказался немного быстрее: среднее время обучения составило около 21.44 секунды против 22.60 секунды у метода с исследовательскими стартами. Разница равна примерно 1.16 секунды, то есть около 5.12%.

По устойчивости относительно основной метрики результаты также близки. Стандартное отклонение ошибки функции ценности у метода с ε -жадной политикой составило примерно 0.739, а у метода с исследовательскими стартами — примерно 0.762.

Метод Монте-Карло с разделенными политиками и взвешенной выборкой по значимости оказался существенно хуже:

$$\text{mean_value_sse} \approx 28874.12, \quad \text{mean_policy_match} \approx 0.507.$$

Следовательно, для данной задачи он оказался менее подходящим. Основная причина состоит в том, что при несовпадении действий поведенческой и целевой политик обратный проход часто обрывается, и значительная часть эпизода не используется для обучения.

Таким образом, лучшим методом для рассматриваемой задачи оказался метод Монте-Карло с единой ε -жадной политикой. Он дал наименьшую ошибку функции ценности, лучшее совпадение с эталонной политикой и немного меньшее время обучения по сравнению с методом с исследовательскими стартами. Метод с исследовательскими стартами также показал хороший результат, однако немного хуже приблизил функцию ценности. Метод с разделенными политиками был исследован, но оказался неэффективным для данной модели очереди.

Заключение

В работе была рассмотрена задача управления очередью в терминах обучения с подкреплением. Состоянием системы являлась длина очереди, а действием — выбор режима работы сервера. Критерий качества учитывал два фактора: штраф за накопление заявок и стоимость выбранного режима обслуживания.

Сначала задача была решена при известной модели переходов. Были реализованы метод итерации по ценности и метод итерации по политикам. Оба метода дали одинаковую оптимальную политику, поэтому функция ценности и политика, полученные методом итерации по ценности, далее использовались как эталонные.

Затем была построена среда-генератор эпизодов `QueueControlEnv`, позволяющая рассматривать модель как «черный ящик». На этой среде были исследованы три метода Монте-Карло: метод с исследовательскими стартами, метод с единой ε -жадной политикой и метод с разделенными политиками и взвешенной выборкой по значимости.

Для сравнения методов использовались две метрики. Основной метрикой была ошибка приближения функции ценности

$$\text{SSE}_V = \sum_{s \in S} (V_{\text{MC}}(s) - V_{\text{VI}}(s))^2.$$

Дополнительно рассматривалась доля состояний, в которых найденная политика совпадает с эталонной политикой Value Iteration.

Лучший результат по основной метрике показал метод Монте-Карло с единой ε -жадной политикой. Для него средняя ошибка функции ценности составила

$$\text{mean_value_sse} \approx 2.83,$$

а среднее совпадение с эталонной политикой составило

$$\text{mean_policy_match} \approx 0.994.$$

Среднее время обучения было около 21.44 секунды.

Метод Монте-Карло с исследовательскими стартами также показал хороший результат, однако немного уступил по основной метрике:

$$\text{mean_value_sse} \approx 3.51, \quad \text{mean_policy_match} \approx 0.980.$$

Его среднее время обучения составило около 22.60 секунды. Таким образом, метод с ε -жадной политикой не только лучше приблизил функцию ценности, но и немного чаще совпадал с эталонной политикой.

Метод с разделенными политиками и взвешенной выборкой по значимости в про-

веденных экспериментах показал существенно худшие результаты:

$$\text{mean_value_sse} \approx 28874.12, \quad \text{mean_policy_match} \approx 0.507.$$

Это связано не с общей непригодностью off-policy подходов, а с особенностями данного алгоритма в выбранной задаче: при несовпадении действия поведенческой политики с целевой обратный проход часто прерывался, поэтому использовалась только небольшая часть эпизода.

Таким образом, для рассматриваемой задачи наиболее удачным оказался метод Монте-Карло с единой ε -жадной политикой. Он обеспечил наименьшую ошибку функции ценности, высокое совпадение с эталонной политикой, достаточную устойчивость результата и небольшое время обучения.

В дальнейшем работу можно продолжить, сравнив методы Монте-Карло с методами временных различий, например Sarsa и Q-обучением. Кроме того, необходимо исследовать другие наборы параметров модели, в том числе такие, при которых оптимальная политика имеет более сложную структуру. Это позволит проверить устойчивость полученных выводов. Также можно усложнить модель очереди: добавить несколько серверов, разные типы заявок, переменную интенсивность поступления или более сложную функцию затрат.

Приложение А. Таблицы вычислительных экспериментов

В приложении приведены таблицы перебора параметров методов Монте-Карло. В основной части работы оставлены графики, выбранные конфигурации и итоговые результаты, а подробные численные значения вынесены в приложение.

В таблицах используются следующие обозначения:

- «Эпизоды» — число обучающих эпизодов;
- «Длина» — максимальная длина одного эпизода;
- `mean_value_sse` — средняя ошибка функции ценности относительно решения Value Iteration;
- `std_value_sse` — стандартное отклонение ошибки функции ценности по случайным запускам;
- `mean_policy_match` — средняя доля состояний, в которых найденная политика совпадает с эталонной политикой;
- `std_policy_match` — стандартное отклонение совпадения политик;
- «Время» — среднее время обучения в секундах.

Основной метрикой сравнения является `mean_value_sse`, поэтому ранг конфигурации в таблицах задается после сортировки по возрастанию этой метрики. Метрика `mean_policy_match` используется как дополнительная.

А.1. Локальный перебор параметров метода Монте-Карло с исследовательскими стартами

В таблице А.1 приведены результаты локального перебора параметров метода Монте-Карло с исследовательскими стартами. Перебор проводился по числу эпизодов, параметру размера шага α и максимальной длине эпизода. Конфигурации отсортированы по основной метрике `mean_value_sse`.

А.2. Локальный перебор параметров метода Монте-Карло с ε -жадной политикой

В таблице А.2 приведены результаты локального перебора параметров метода Монте-Карло с ε -жадной политикой. Перебор проводился по числу эпизодов, параметру размера шага α , параметру исследования ε и максимальной длине эпизода. Конфигурации также отсортированы по возрастанию ошибки функции ценности.

А.3. Перебор параметров метода Монте-Карло с разделенными политиками

В таблице А.3 приведены результаты грубого перебора параметров метода Монте-Карло с разделенными политиками и взвешенной выборкой по значимости. Перебор проводился по числу эпизодов, параметру исследования поведенческой политики ϵ и максимальной длине эпизода.

Таблица А.1: Локальный перебор параметров: метод с исследовательскими стартами

Ранг	Эпизоды	α	Длина	mean_value_sse	std_value_sse	mean_policy_match	std_policy_match	Время	СКО времени
1	3000	0.015	150	3.514	0.762	0.980	0.000	22.60	0.418
2	3000	0.015	200	3.817	0.869	0.988	0.012	29.79	0.178
3	2000	0.015	200	4.335	0.945	0.982	0.011	19.92	0.170
4	2000	0.015	150	4.369	1.642	0.975	0.009	15.09	0.157
5	3000	0.020	150	4.718	1.450	0.986	0.009	22.78	0.211
6	1500	0.015	200	4.934	1.887	0.979	0.014	16.10	1.293
7	2000	0.020	150	5.081	0.870	0.982	0.011	15.17	0.247
8	3000	0.020	200	5.217	1.392	0.986	0.009	30.47	0.243
9	1500	0.020	200	5.262	1.122	0.975	0.013	15.82	0.199
10	3000	0.015	100	5.338	0.823	0.971	0.013	15.59	0.589
11	2000	0.020	200	5.493	0.992	0.982	0.011	20.17	0.169
12	1500	0.015	150	5.828	2.736	0.968	0.013	10.23	0.175
13	3000	0.020	100	6.120	1.028	0.973	0.018	14.89	0.104
14	2000	0.015	100	6.145	1.190	0.969	0.020	10.08	0.085
15	2000	0.020	100	6.278	1.688	0.975	0.009	9.98	0.077
16	1500	0.020	150	6.445	1.962	0.979	0.014	12.05	0.157
17	2000	0.030	150	6.949	0.967	0.984	0.008	15.08	0.125
18	1500	0.015	100	7.216	2.080	0.950	0.019	6.59	0.087
19	1500	0.020	100	7.246	2.210	0.977	0.008	8.42	0.168
20	3000	0.030	150	7.285	1.883	0.984	0.012	23.46	0.117

Таблица А.2: Локальный перебор параметров: метод с ε -жадной политикой

Ранг	Эпизоды	α	ε	Длина	mean_value_sse	std_value_sse	mean_policy_match	std_policy_match	Время	СКО времени
1	3000	0.010	0.01	150	2.829	0.739	0.994	0.009	21.44	0.188
2	3000	0.010	0.02	150	2.830	0.906	0.994	0.009	22.04	0.149
3	2000	0.010	0.01	200	2.833	0.747	0.992	0.014	19.29	0.086
4	3000	0.010	0.01	200	2.942	0.955	0.988	0.019	28.85	0.344
5	3000	0.010	0.03	200	3.031	0.962	0.996	0.008	32.99	2.594
6	2000	0.010	0.02	200	3.082	1.031	0.996	0.008	19.74	0.539
7	2000	0.010	0.01	150	3.190	1.491	0.994	0.009	14.54	0.173
8	1500	0.010	0.01	200	3.251	0.806	0.988	0.014	17.42	0.612
9	3000	0.010	0.03	150	3.317	1.232	0.996	0.008	24.11	1.822
10	1500	0.010	0.02	200	3.359	0.836	0.988	0.017	15.60	0.772
11	2000	0.010	0.03	200	3.389	1.377	0.994	0.009	19.48	0.301
12	3000	0.010	0.02	200	3.391	1.596	0.996	0.008	29.42	0.238
13	2000	0.010	0.02	150	3.442	1.296	0.992	0.010	14.78	0.497
14	2000	0.010	0.03	150	3.505	1.369	0.992	0.014	15.58	0.292
15	3000	0.015	0.02	150	3.650	1.209	0.998	0.006	21.86	0.196
16	1500	0.010	0.02	150	3.693	0.864	0.988	0.014	11.92	0.366
17	3000	0.010	0.03	100	3.796	1.144	0.994	0.013	14.81	0.194
18	1500	0.010	0.03	200	3.809	1.079	0.992	0.010	14.73	0.113
19	2000	0.015	0.02	150	3.894	0.640	0.998	0.006	14.52	0.111
20	1500	0.010	0.03	150	3.923	1.474	0.986	0.016	11.17	0.208

Таблица А.3: Перебор параметров: метод с разделенными политиками

Ранг	Эпизоды	ε	Длина	mean_value_sse	std_value_sse	mean_policy_match	std_policy_match	Время	СКО времени
1	50000	0.15	20	28874.12	1481.05	0.507	0.029	44.73	0.303
2	50000	0.15	50	29732.14	1152.64	0.611	0.111	110.69	0.906
3	50000	0.10	20	29837.41	1844.96	0.520	0.064	43.64	0.444
4	50000	0.15	30	30602.62	1500.71	0.546	0.087	66.05	0.524
5	50000	0.20	30	30811.77	1667.46	0.559	0.079	69.86	0.630
6	50000	0.20	50	30963.82	932.07	0.569	0.078	116.74	1.336
7	50000	0.10	30	31253.32	1890.84	0.572	0.065	65.42	0.679
8	50000	0.20	20	31529.60	1245.93	0.533	0.067	45.67	0.357
9	50000	0.10	50	31803.21	1000.23	0.592	0.085	110.47	1.223
10	20000	0.15	20	37237.22	2044.96	0.477	0.078	17.43	0.110
11	20000	0.15	50	38311.39	1878.14	0.507	0.061	42.74	0.606
12	20000	0.10	20	38818.52	2218.03	0.487	0.115	17.31	0.284
13	20000	0.20	50	39461.16	849.36	0.490	0.093	43.73	0.545
14	20000	0.15	30	39627.28	1652.39	0.431	0.088	25.63	0.214
15	20000	0.10	30	39841.60	2049.89	0.444	0.165	25.68	0.345
16	20000	0.20	30	39896.98	2064.24	0.477	0.073	26.34	0.426
17	20000	0.10	50	40389.46	1371.03	0.412	0.048	44.67	0.690
18	20000	0.20	20	41048.13	1869.07	0.438	0.089	17.60	0.153